

Loop Control — break & continue

Four problems. For each: the thinking process, the flowchart, then the code.

- break — stop loop
- continue — skip iteration
- push — stack operation
- data / output

PROBLEM 01

Exam Score Processor

Process exam scores from an ArrayList. Push passing scores (≥ 50) onto a Stack. Skip any score equal to 0 (absent student). Stop completely if a score of 100 is found.

THINK FIRST — PEN & PAPER

Data

ArrayList of ints, e.g. [72, 0, 45, 88, 55, 0, 100, 91]

STOP?

score == 100 → break out of the loop entirely

SKIP?

score == 0 → continue to next element

DO?

score ≥ 50 → push onto stack. Otherwise ignore.

Order

break guard first → continue guard second → logic last.

FLOWCHART

```
import java.util.*;

public class GradesShorter {
    public static void main(String[] args) {

        ArrayList<Integer> grades = new ArrayList<>(  
            Arrays.asList(45, 78, 0, 92, 100, 99, 30));

        Stack<Integer> passed = new Stack<>();

        for (int score : grades) {  
            // 1. STOP guard — always first  
            if (score == 100) { break; }

            // 2. SKIP guard — always second  
            if (score == 0) { continue; }

            // 3. LOGIC — only runs for valid, in-range scores  
            if (score >= 50) { passed.push(score); }
        }

        System.out.println("Passed: " + passed);
    }
}
```

PROBLEM 02

Product Inventory Processor

Iterate a Hashtable of product names and quantities. Skip any product with quantity == 0 (out of stock). Stop when a quantity greater than 100 is found.

THINK FIRST — PEN & PAPER

Data

Hashtable<String, Integer> — need both name and quantity

Iterate

Use Map.Entry to get key and value in the same step

STOP?

qty > 100 → break (oversized quantity, halt scan)

SKIP?

qty == 0 → continue (out of stock, skip)

DO?

Print product name and quantity normally

FLOWCHART

```
import java.util.*;

public class ProductsBetter {
    public static void main(String[] args) {

        Hashtable<String, Integer> products = new Hashtable<>();
        products.put("DC motor", 12);
        products.put("Resistors", 4);
        products.put("Phones", 0);
        products.put("Capacitors", 101);

        // Map.Entry gives BOTH key and value in one step
        for (Map.Entry<String, Integer> entry : products.entrySet()) {
            String name = entry.getKey();
            int quantity = entry.getValue();

            // 1. STOP guard
            if (quantity > 100) {
                System.out.println("Stopping at: " + name);
                break;
            }

            // 2. SKIP guard
            if (quantity == 0) {
                System.out.println("Skipping: " + name);
                continue;
            }

            // 3. LOGIC
            System.out.printf("%-15s qty = %d\n", name, quantity);
        }
    }
}
```

PROBLEM 03

Day of Week Enum

Use a Java enum for the days of the week. Iterate through all values, skip MONDAY, and stop processing when THURSDAY is reached.

THINK FIRST — PEN & PAPER

Data

enum.values() returns an array — use for-each

Tip

Enums are singletons — compare with ==, not .equals()

STOP?

day == THURSDAY → break (THURSDAY itself is not printed)

SKIP?

day == MONDAY → continue (skip it, move on)

DO?

Print the day name

FLOWCHART

```
public class DaysOfWeek {

    // Enum defined inside the class — fine when used in one file only
    enum WeekDays {
        SATURDAY, SUNDAY, MONDAY, TUESDAY,
        WEDNESDAY, THURSDAY, FRIDAY
    }

    public static void main(String[] args) {

        for (WeekDays day : WeekDays.values()) {

            // 1. STOP guard — use == for enum comparison
            if (day == WeekDays.THURSDAY) { break; }

            // 2. SKIP guard
            if (day == WeekDays.MONDAY) { continue; }

            // 3. LOGIC
            System.out.println(day);
        }
    }
}
```

PROBLEM 04

Even Number Stack & Sum

Process an ArrayList of integers. Skip negative numbers. Stop when a number > 50 is found. Push only even numbers onto a Stack. After processing, pop everything and calculate the sum.

THINK FIRST — PEN & PAPER

Data

e.g. [-1, 42, 32, 3, 51, 100, 99, 30]

STOP?

number > 50 → break immediately

SKIP?

number < 0 → continue (negatives ignored)

DO?

number % 2 == 0 → push. Odd numbers silently ignored.

After

while stack not empty — pop and add to running sum

FLOWCHART

```
import java.util.*;

public class StackExampleBetter {
    public static void main(String[] args) {

        ArrayList<Integer> numbers = new ArrayList<>(  
            Arrays.asList(-1, 42, 32, 3, 51, 100, 99, 30));

        Stack<Integer> processedNumbers = new Stack<>();

        // — Phase 1: fill the stack —————
        for (int number : numbers) {

            // 1. STOP guard
            if (number > 50) { break; }

            // 2. SKIP guard — saves the even-check below
            if (number < 0) { continue; }

            // 3. LOGIC
            if (number % 2 == 0) { processedNumbers.push(number); }
        }

        // — Phase 2: drain the stack —————
        int sum = 0;
        while (!processedNumbers.empty()) {
            sum += processedNumbers.pop();
        }

        System.out.println("Sum of even stack elements: " + sum);
        // 51 triggers break — stack has [42, 32] → sum = 74
    }
}
```

QUICK REFERENCE

The Three-Question Rule

Ask these three questions about every element, in this exact order, every single time.

① STOP?

Should I halt the entire loop because of this element?

→ break;

② SKIP?

Should I ignore this element and jump to the next one?

→ continue;

③ DO?

What is the actual logic for valid, in-range elements?

→ your logic

universal pattern — apply to every loop you write

```
for (T element : collection) {  
  
    if (skipCondition) { break; } // ① always first  
    if (skipCondition) { continue; } // ② always second  
  
    // ③ logic — only reached by clean, valid elements  
    doSomethingWith(element);  
}
```